

BTS Services Informatiques aux Organisations (SIO)

Session 2026

Nom du candidat : SEILLER

Prénom du candidat : Julian

Parcours : SLAM

Fiche de réalisation professionnelle N°8

Déploiement de l'environnement de développement via

Docker et PostgreSQL

(Projet CardiacTS)

Période de réalisation : Du lundi 2 mars au vendredi 3 avril

Type de réalisation : En stage (Laboratoire TIMC)

Mode de réalisation : En équipe (Binôme)

Sommaire

Contexte Organisationnel.....	3
Problématique de gestion.....	3
Choix de la solution mise en œuvre.....	3
Réalisation de la solution.....	4
a - Prérequis et ressources à disposition.....	4
b - Mise en œuvre de la solution.....	4
c - Tests.....	4
i - Cas d'utilisation 1 : Lancement de l'infrastructure.....	4
ii - Cas d'utilisation 2 : Connexion depuis l'application	
Java.....	5
iii - Cas d'utilisation 3 : Persistance des données.....	5
Conclusions.....	5
Annexes.....	5
Compétences associées.....	6

Contexte Organisationnel

Cette mission s'est déroulée au **Laboratoire TIMC** (équipe **BCM**). Dans le cadre du projet **CardiacTS**, nous devons travailler à deux (en binôme) sur le développement de l'application Java. Pour garantir que le logiciel fonctionne de la même manière sur nos deux postes de travail, il était indispensable de standardiser notre environnement technique.

Problématique de gestion

Le développement d'une application liée à une base de données

PostgreSQL pose souvent des problèmes de configuration :

- **Incohérence des versions** : Si un développeur utilise PostgreSQL 14 et l'autre la version 16, des erreurs de compatibilité peuvent apparaître.
- **Complexité d'installation** : Installer et configurer manuellement un serveur SQL sur chaque machine est long et sujet aux erreurs (gestion des ports, des droits utilisateurs, des variables d'environnement).
- **Pollution du système** : Installer de nombreux services directement sur l'OS peut ralentir la machine et créer des conflits avec d'autres projets. L'enjeu était de pouvoir lancer l'intégralité de l'infrastructure de données en une seule commande, de manière identique sur n'importe quel ordinateur.

Choix de la solution mise en œuvre

Nous avons choisi d'utiliser la technologie de **conteneurisation Docker** :

- **Docker Desktop / Engine** : Pour isoler le service de base de données du système d'exploitation hôte.
- **Docker Compose** : Pour définir l'infrastructure sous forme de code (**Infrastructure as Code**). Un simple fichier de configuration permet de décrire le service PostgreSQL, le réseau et le stockage.
- **Image officielle PostgreSQL** : Utilisation d'une image certifiée pour garantir la sécurité et la stabilité de l'environnement.

Réalisation de la solution

a - Prérequis et ressources à disposition

- Logiciel **Docker Desktop** installé sur les postes de développement.
- IDE **IntelliJ IDEA** avec le plugin Docker.
- Accès au dépôt **GitHub** pour partager les fichiers de configuration entre binômes.

b - Mise en œuvre de la solution

- **Rédaction du docker-compose.yml** : Nous avons conçu le fichier de configuration définissant le service **db**. J'ai spécifié l'image (**postgres:latest**), le nom du conteneur, et le mappage des ports (5432:5432).
- **Gestion des variables d'environnement** : Pour respecter les bonnes pratiques de sécurité, les identifiants POSTGRES_USER ; POSTGRES_PASSWORD ont été externalisés dans un fichier **.env**, non suivi par Git.
- **Persistance des données** : J'ai configuré un **Volume Docker**. Cela permet aux données de la base de rester enregistrées sur le disque dur même si le conteneur est arrêté ou supprimé.
- **Initialisation** : Création d'un script SQL de structure (init.sql) placé dans le dossier « docker-entrpoint-initdb.d » pour créer automatiquement les tables (patients, imageries, logs) au premier lancement.

c - Tests

i - Cas d'utilisation 1 : Lancement de l'infrastructure

- **Entrée / Action** : Exécution de la commande « docker-compose up -d » dans le terminal.
- **Résultat attendu** : Docker télécharge l'image, crée le réseau et lance le conteneur. Le statut passe au vert (Running) dans l'interface Docker Desktop.

ii - Cas d'utilisation 2 : Connexion depuis l'application Java

- **Entrée / Action** : Lancement de l'application **CardiacTS** configurée pour pointer sur localhost:5432.
- **Résultat attendu** : Le pool de connexions **HikariCP** parvient à établir le lien avec la base de données conteneurisée. Les données s'affichent dans l'interface JavaFX.

iii - Cas d'utilisation 3 : Persistance des données

- **Entrée / Action** : Arrêt du conteneur, suppression de celui-ci, puis relance via Docker Compose.
- **Résultat attendu** : Grâce au **Volume**, les dossiers patients créés lors de la session précédente sont toujours présents.

Conclusions

L'utilisation de **Docker** a radicalement simplifié notre flux de travail en binôme. Nous avons éliminé le fameux problème du "ça marche sur ma machine mais pas sur la tienne". Cette approche m'a permis de comprendre les bénéfices de la **virtualisation légère** et de l'automatisation des déploiements, des compétences indispensables pour le travail en équipe et la mise en production moderne.

Annexes

- **Annexe 1 :** Capture d'écran du fichier `docker-compose.yml` utilisé pour le projet.

```
services:
  postgres:
    image: postgres:14-alpine
    container_name: cardiacts-postgres
    restart: always
    environment:
      POSTGRES_DB: cardiacts_db
      POSTGRES_USER: cardiacts_admin
      POSTGRES_PASSWORD: super_password_123
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

  minio:
    image: minio/minio
    container_name: cardiacts-minio
    restart: always
    environment:
      MINIO_ROOT_USER: minio_admin
      MINIO_ROOT_PASSWORD: minio_password_123
    command: server /data --console-address ":9001"
    ports:
      - "9000:9000"
      - "9001:9001"
    volumes:
      - minio_data:/data

volumes:
  postgres_data:
  minio_data:
```

- **Annexe 2 :** Vue de l'interface **Docker Desktop** montrant le conteneur PostgreSQL en cours d'exécution.

Compétences associées

N°	Compétences associées
B1.5.2	Mettre à disposition des utilisateurs un service informatique
B1.1.2	Déployer un environnement d'exploitation et de développement
B1.4.2	Installer et configurer des éléments d'infrastructure (Docker/SQL)
B1.4.2	Travailler en équipe informatique (Partage de config via Git)